

Accso Technical Interview Assignment

You are joining a client engagement as a software consultant for an e-commerce platform. The company ships orders through multiple courier partners.

The problem: downstream systems disagree about shipment status because courier events arrive late, out of order, duplicated, and sometimes with conflicting data. Customer support, order tracking, and incident response all depend on having a reliable internal view of what's actually happening with each shipment.

Your task is to build a focused backend service to solve this. Don't aim for a large platform. Show us your engineering judgment: how you frame the problem, choose scope, implement the hard parts well, handle change, and explain your trade-offs. State your assumptions explicitly and surface uncertainty rather than making silent assumptions.

Before you start

We encourage AI coding assistants and agentic tooling. We will evaluate how you used them: where they helped, where they were weak or wrong, what you changed or rejected, and how you stayed in control of the work.

Be specific. "I used Copilot for boilerplate" doesn't tell us much. "AI suggested an in-memory deduplication approach; I rejected it because it wouldn't survive restarts and replaced it with a database constraint" does.

Target around **4–6 hours** of focused work. If you spend significantly more or less, note it.

Domain context

The client receives shipment events from courier partners via webhook. Internal systems need two things:

1. A reliable current view of each shipment: what's the status right now?
2. A queryable history of what happened and how we got to the current view

Example inbound event:

```
{
  "eventId": "evt-123",
  "partner": "dhl",
  "shipmentId": "ship-456",
  "status": "IN_TRANSIT",
  "occurredAt": "2026-03-10T12:00:00Z",
  "receivedAt": "2026-03-10T12:00:05Z",
  "location": "Amsterdam"
}
```

Adjust this schema if a different shape better solves the problem. Explain why if you do.

Suggested status values:

- LABEL_CREATED
- HANDED_TO_CARRIER
- IN_TRANSIT
- OUT_FOR_DELIVERY
- DELIVERED
- DELIVERY_EXCEPTION
- RETURNED

The client prioritizes handling duplicates, out-of-order events, and conflicting updates over feature breadth. Keep that in mind throughout.

Functional requirements

Implement a minimal API with these capabilities:

1. Ingest shipment events

POST /shipment-events

Expectations:

- Persist the inbound event or a normalized representation of it
- Detect duplicate events using a rule you define and document
- Preserve an audit trail of all accepted events
- Return a clear response indicating whether the event was accepted, rejected as a duplicate, or caused a state change

2. Query current shipment state

GET /shipments/{shipmentId}

Minimum response:

- Shipment ID
- Current status
- When that status occurred (the `occurredAt` timestamp, not `receivedAt`)
- How many events have been processed for this shipment
- A brief explanation of why this is the current state, if it's not obvious from your model

3. Query shipment history

GET /shipments/{shipmentId}/events

Expectations:

- Show events in a clear order (explain your ordering choice)
- Make it clear how you handled duplicates or out-of-order events (accepted, rejected, reordered, etc.)

Data integrity rules

Your solution must document explicit decisions for these cases:

1. **Duplicate events:** The same event is received more than once
2. **Out-of-order events:** An older event arrives after a newer one has already been processed
3. **Conflicting updates:** Two events from different sources or with different data describe the same moment

There's no single correct approach. We care that your rules are coherent, explicit, and testable. A simple approach you can defend beats a sophisticated one you can't explain.

Change request

After you have a working initial solution, **commit it to git before continuing**. Then apply this change:

A new courier partner cannot provide a stable eventId. They resend the same update multiple times, sometimes with different receivedAt values. The client still expects your service to avoid obvious duplicates while preserving auditability.

Implement this fully or implement the most important slice and document the remainder. The key question: **does your original design absorb this change cleanly, or does it break?**

In your documentation, explain:

- What changed in your code, model, or rules
 - What stayed the same and why
 - What you would still need to handle if this went to production
-

Required deliverables

Submit a repository containing:

1. **A working solution** with clear run instructions. Another engineer should be able to clone and run it within 5 minutes.
2. **A README** covering: problem framing (how you interpreted the brief), assumptions you made, design choices and trade-offs (why this approach over alternatives you considered), known limitations, and what changed when you applied the change request.
3. **2 short ADRs** (Architecture Decision Records) covering your two most important design choices. Name the decision, list the alternatives you considered, and explain why you chose what you chose.
4. **Automated tests** covering: decision logic (your data integrity rules), at least one integration or end-to-end path, and the change request scenario.

5. **A development process note** describing how you used AI tools: what you used, what it produced, what you overrode or verified yourself, and at least one concrete example where your judgment diverged from the AI's suggestion.

Optional bonus: Include lightweight observability (structured logs, counters, or a simple health endpoint) if it meaningfully supports your design. Don't add it just to look complete.

Keep packaging lightweight. Containerization (e.g., Docker Compose) is welcome but not required. Don't add auth stacks, cloud deployment, frontends, CI pipelines, or other artifacts that don't serve the problem.

Interview discussion

In the follow-up interview, expect to discuss your assumptions, scoping decisions, data integrity rules and their edge cases, how the change request affected your design, your use of AI tools, and what you'd do next with more time.